

# Additional Questions and Answers related to Lecture 5

**M.G.Noel A.S Fernando (PhD)**

**Professor**

**NSBM/Dept. of Information Systems Engineering, UCSC**



# Question 1

- Insert the following values in order :10, 20, 5, 6, 12, 30, 7, 17 (order is 3-4)
- We want to create a **3–4 B-tree** (also called a 2–3–4 tree) using the data set:
- 10, 20, 5, 6, 12, 30, 7, 17
- A 3–4 tree node can have **2, 3, or 4 children**, and **1–3 keys**.
- **Step 1:** Insert 10
- Tree is empty → 10 becomes the root.
- [10]

# Question 1 cont..

- Step 2: Insert 20
- Root has 1 key  $\rightarrow$  can accommodate another  $\rightarrow$  insert 20.
- [10, 20]
- Step 3: Insert 5
- Root has 2 keys  $\rightarrow$  can accommodate another  $\rightarrow$  insert 5 in order.
- [5, 10, 20]

# Question 1 cont..

- Step 4: Insert 6
  - Root has 3 keys → needs to **split** when inserting a 4th key.
  - Order keys: 5, 6, 10, 20
  - Middle key **10** moves up → becomes new root
  - Left child: [5, 6], Right child: [20]

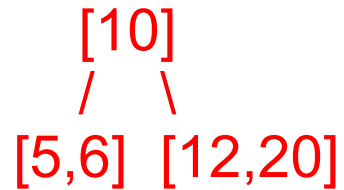
[10]

/ \

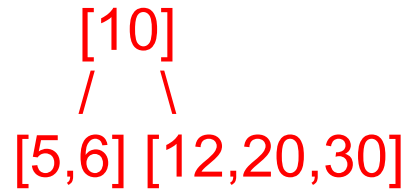
[5,6] [20]

# Question 1 Cont..

- Step 5: Insert 12  
12 > 10 → goes to right child [20]  
Right child has 1 key → can accommodate → insert 12 in order → [12, 20]



- Step 6: Insert 30
  - 30 > 10 → goes to right child [12, 20]
  - Right child has 2 keys → can accommodate → insert 30 → [12, 20, 30]



# Question 1 Cont..

- Step 7: Insert 7
  - $7 < 10 \rightarrow$  goes to left child  $[5,6]$
  - Left child has 2 keys  $\rightarrow$  can accommodate  $\rightarrow$  insert 7  $\rightarrow [5,6,7]$

[10]

/   \

[5,6,7] [12,20,30]

# Question 1 Cont..

- Step 8: Insert 17
  - $17 > 10 \rightarrow$  goes to right child  $[12, 20, 30]$
  - Right child has 3 keys  $\rightarrow$  will **split**
  - Order:  $12, 17, 20, 30 \rightarrow$  middle key 20 moves up to root
  - Root  $[10]$  becomes  $[10, 20]$
  - Children: Left  $\rightarrow [12, 17]$ , Right  $\rightarrow [30]$
  - Left child remains  $[5, 6, 7]$

$[10, 20]$

/   |   \

$[5, 6, 7]$   $[12, 17]$   $[30]$

## Question 2

- Insert the following numbers into an initially empty B-tree to build a B-tree of order( $m$ ) = 4
- [5,3,21,9,13,22,7,10,11,14,8,16]



## Answer to the question 2

- **Step 1** – Calculate the maximum  $(m-1)$  and, minimum  $(\lceil m/2 \rceil - 1)$  number of keys a node can hold, where  $m$  is denoted by the order of the B Tree

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

- Order  $(m) = 4$
- Maximum Keys  $(m - 1) = 3$
- Minimum Keys  $(\lceil \frac{m}{2} \rceil) - 1 = 1$
- Maximum Children  $= 4$
- Minimum Children  $(\lceil \frac{m}{2} \rceil) = 2$

# Answer to the question Cont....

- **Step 2** – The data is inserted into the tree using the binary search insertion and once the keys reach the maximum number, the node is split into half and the median key becomes the internal node while the left and right keys become its children.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 9 will cause overflow in the node; hence it must be split.

**Step 3** – All the leaf nodes must be on the same level.

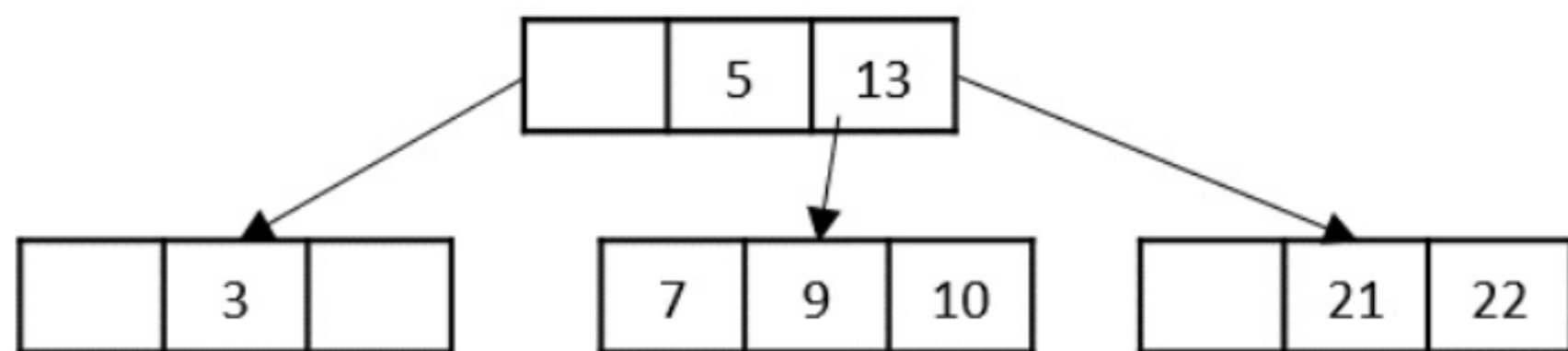
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 22 will cause overflow in the node; hence it must be split.

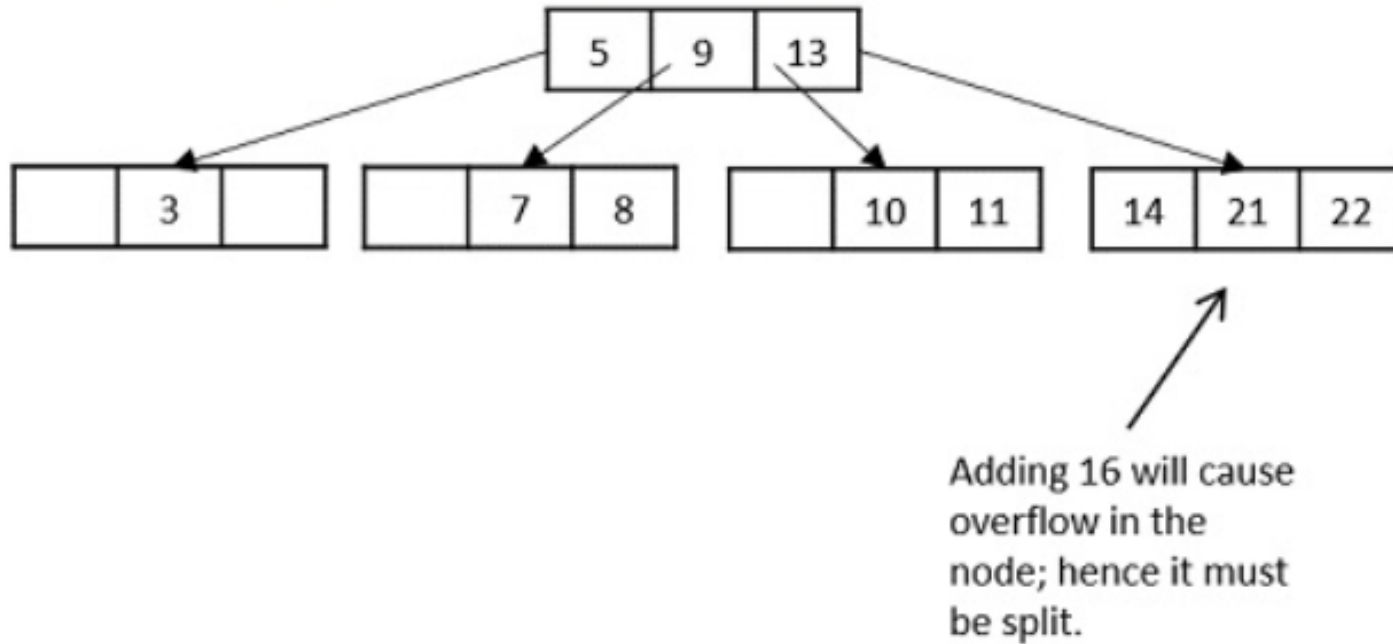
The keys, 5, 3, 21, 9, 13 are all added into the node according to the binary search property but if we add the key 22, it will violate the maximum key property. Hence, the node is split in half, the median key is shifted to the parent node and the insertion is then continued.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



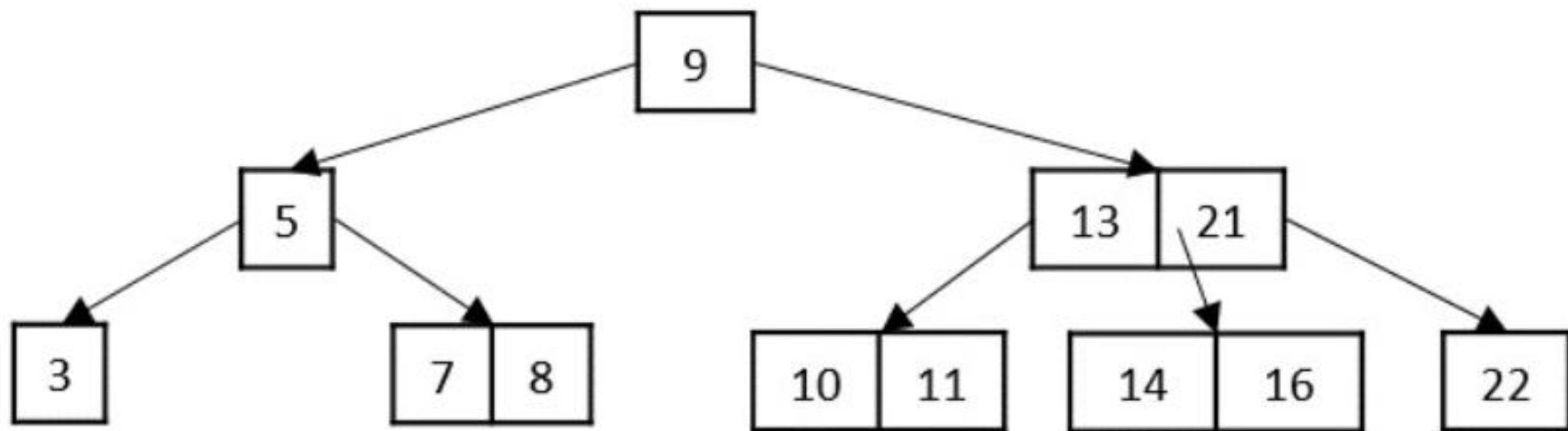
Adding 11 will cause  
overflow in the  
node; hence it must  
be split.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



While inserting 16, even if the node is split in two parts, the parent node also overflows as it reached the maximum keys. Hence, the parent node is split first and the median key becomes the root. Then, the leaf node is split in half the median of leaf node is shifted to its parent.

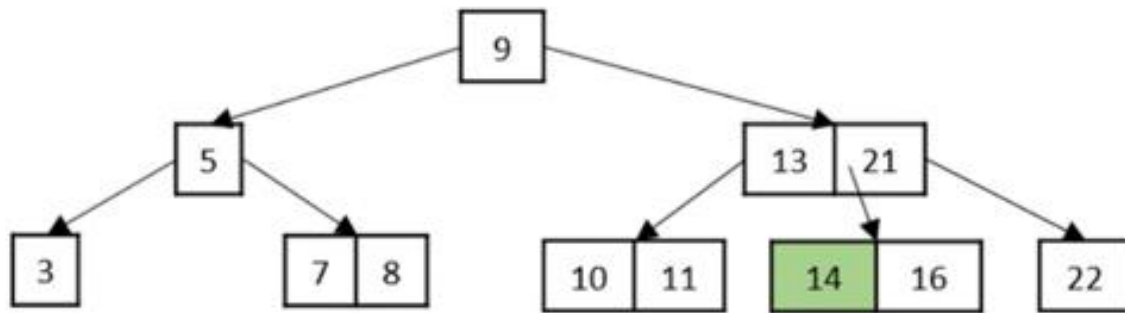
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



The final B tree after inserting all the elements is achieved.

# Question 3

- Consider the following Tree



**Perform the following operations:**

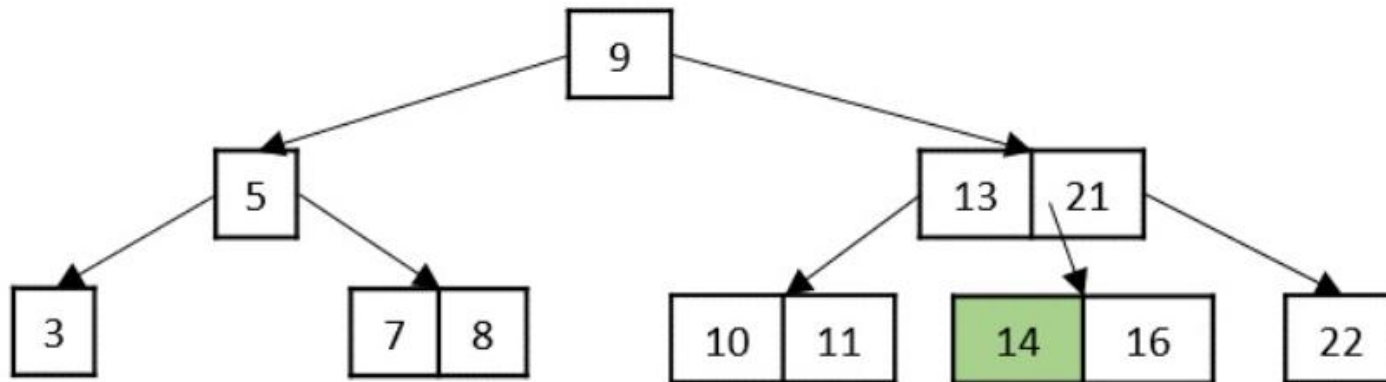
- 1) Delete node 14
- 2) Delete node 3
- 3) Delete node 13
- 4) Delete node 5

- Note:** All deletions should be performed on the original tree.

# Deletion operation

1 – If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.

Delete key 14

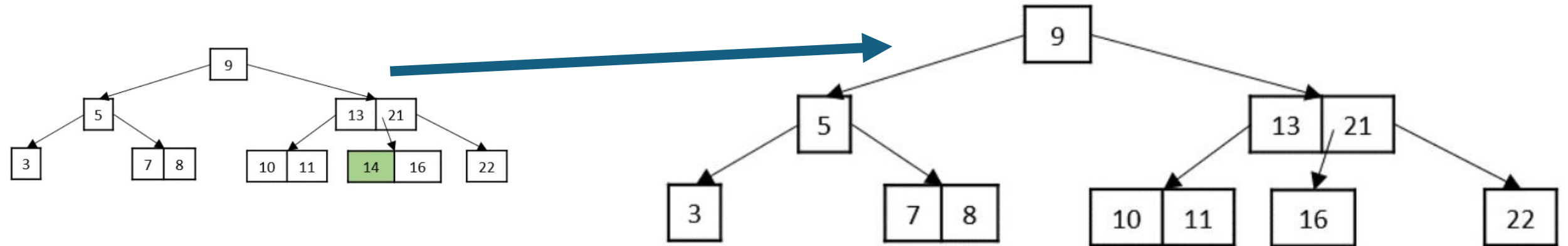




# Deletion operation

1) – If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.

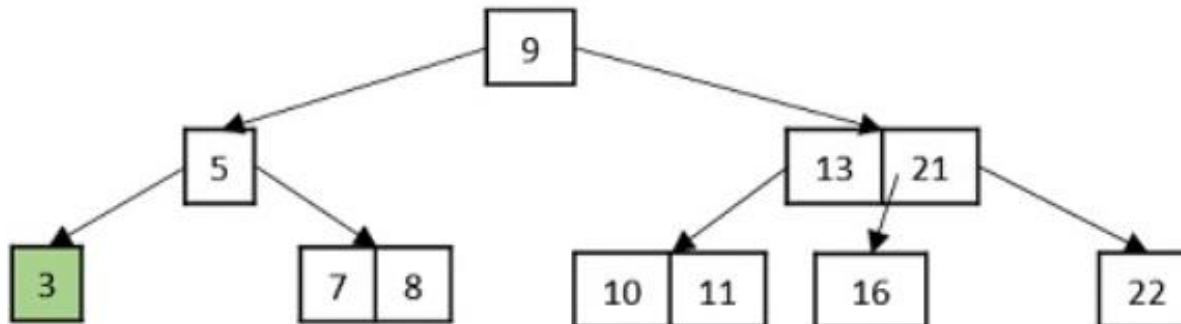
Delete key 14



# Deletion operation (cont..)

**2)** – If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling.

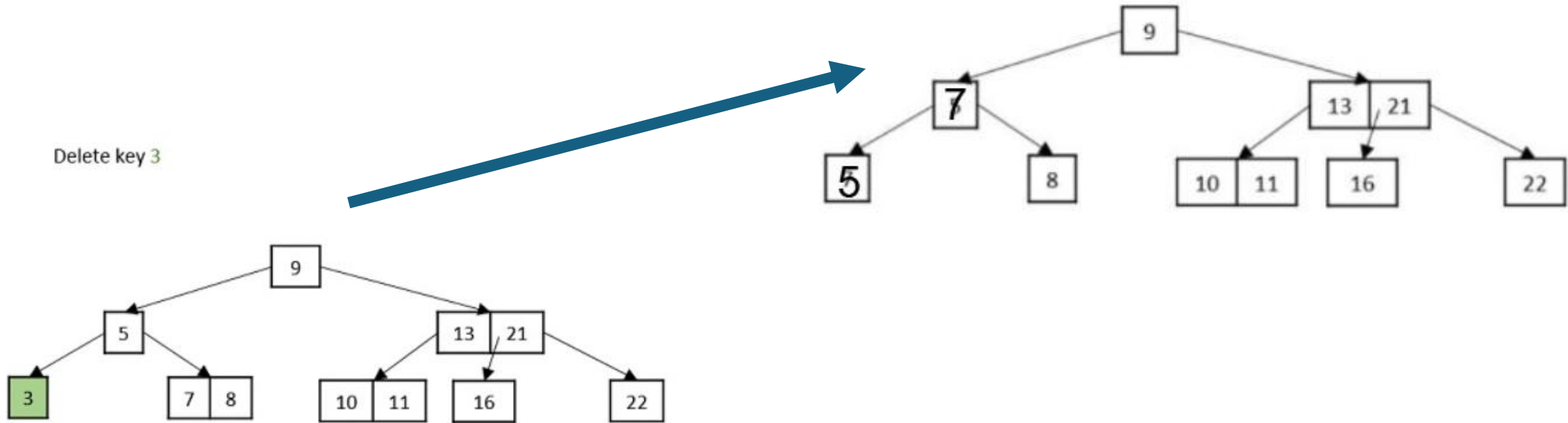
Delete key 3



# Deletion operation (cont..)

2)

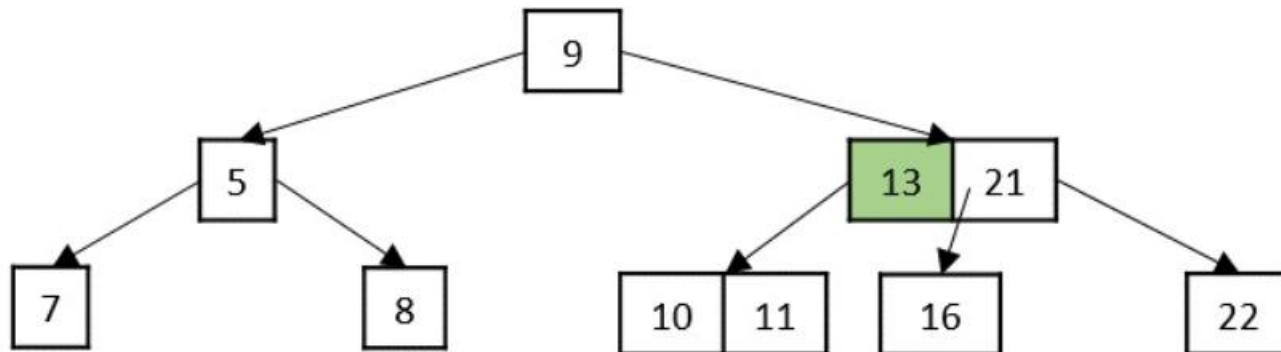
Delete key 3



# Deletion operation (cont..)

**3)** If the key to be deleted is in an internal node, it is replaced by a key in either left child or right child based on which child has more keys. But if both child nodes have minimum number of keys, they are merged together.

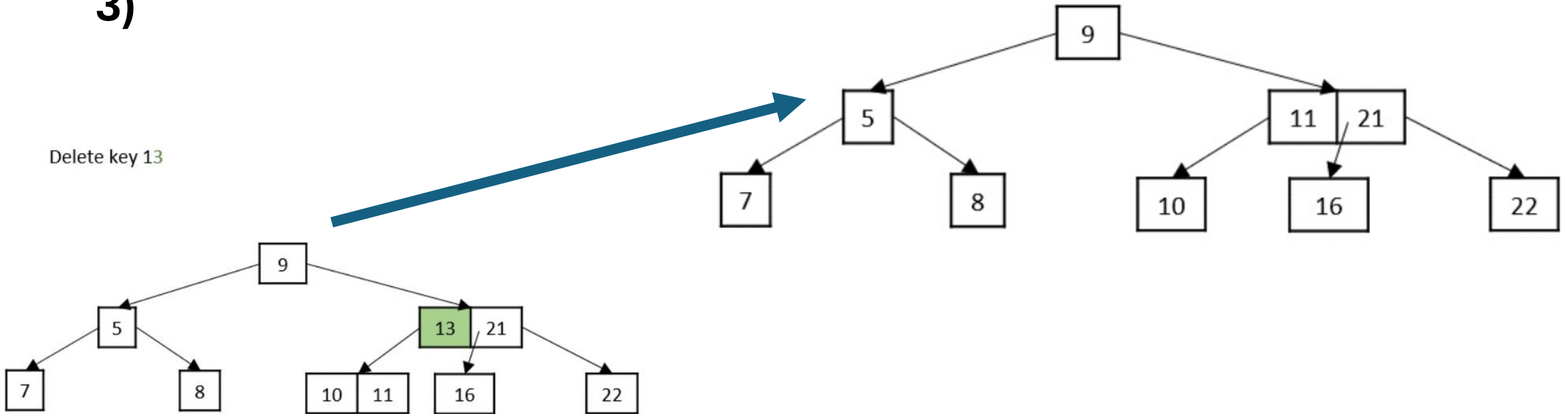
Delete key 13



# Deletion operation (cont..)

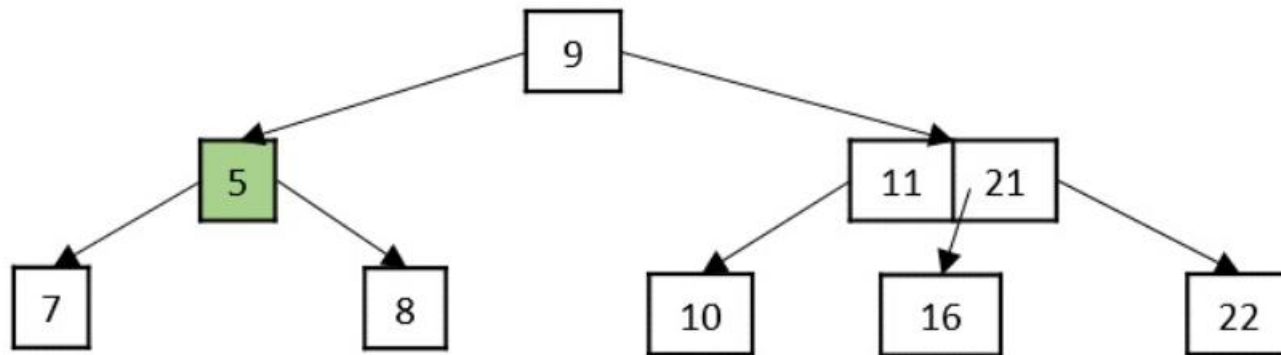
3)

Delete key 13



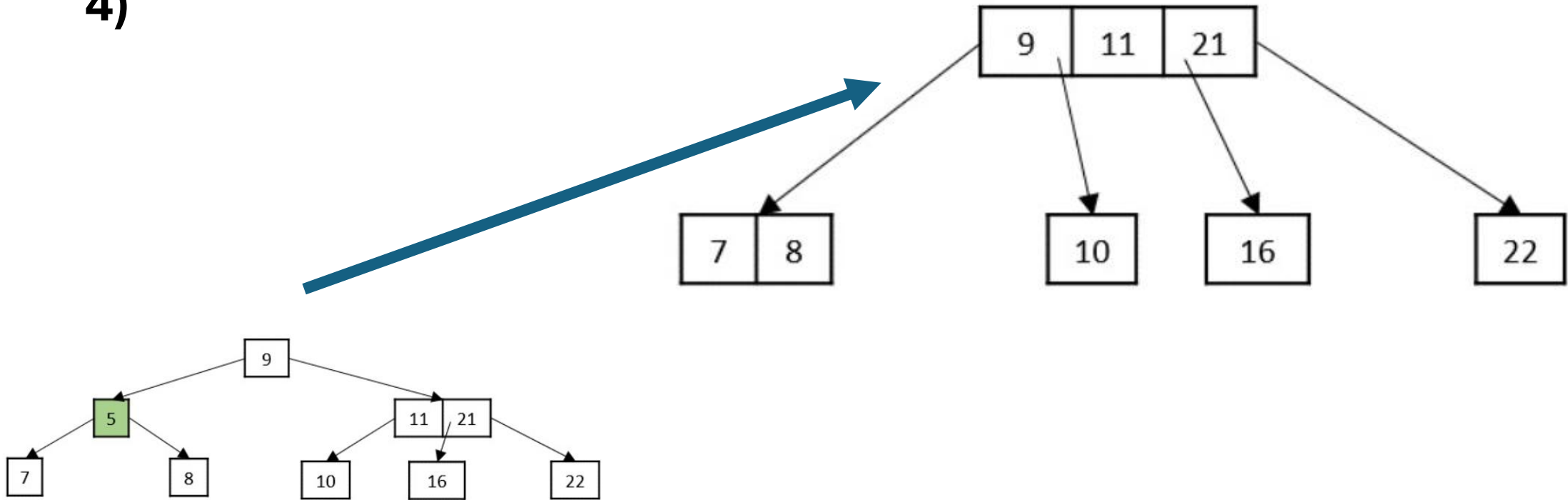
## Deletion operation (cont..)

4 )– If the key to be deleted is in an internal node violating the minimum keys property, and both its children and sibling have minimum number of keys, merge the children. Then merge its sibling with its parent.



# Deletion operation (cont..)

4)



## Question 4

- Consider the following set of numbers:  
**10, 20, 5, 6, 12, 30, 7, 17**
- **a)** Create a B+ Tree using the above dataset when order  $(m)=3$  .  
**Note:** You may present the step-by-step procedure of B+ Tree construction, including explanations at each intermediate step.
- **b)** Why are the leaf nodes linked in a B+ Tree?

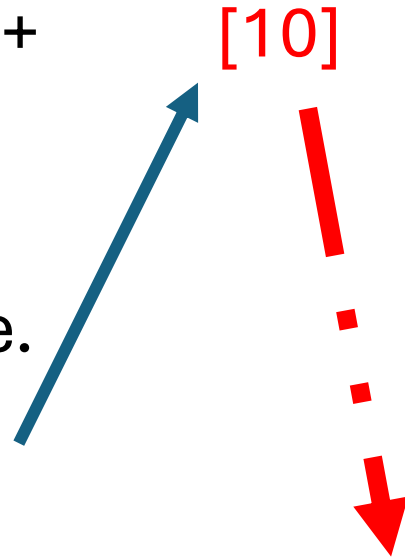


# Answer : Question 4

- Step-by-Step Insertion in B+ Tree (Order = 3)

- **Step 1: Insert 10**

- Start with an empty B+ Tree.
- Insert 10 in the **leaf node**.



- **Step 2: Insert 20**

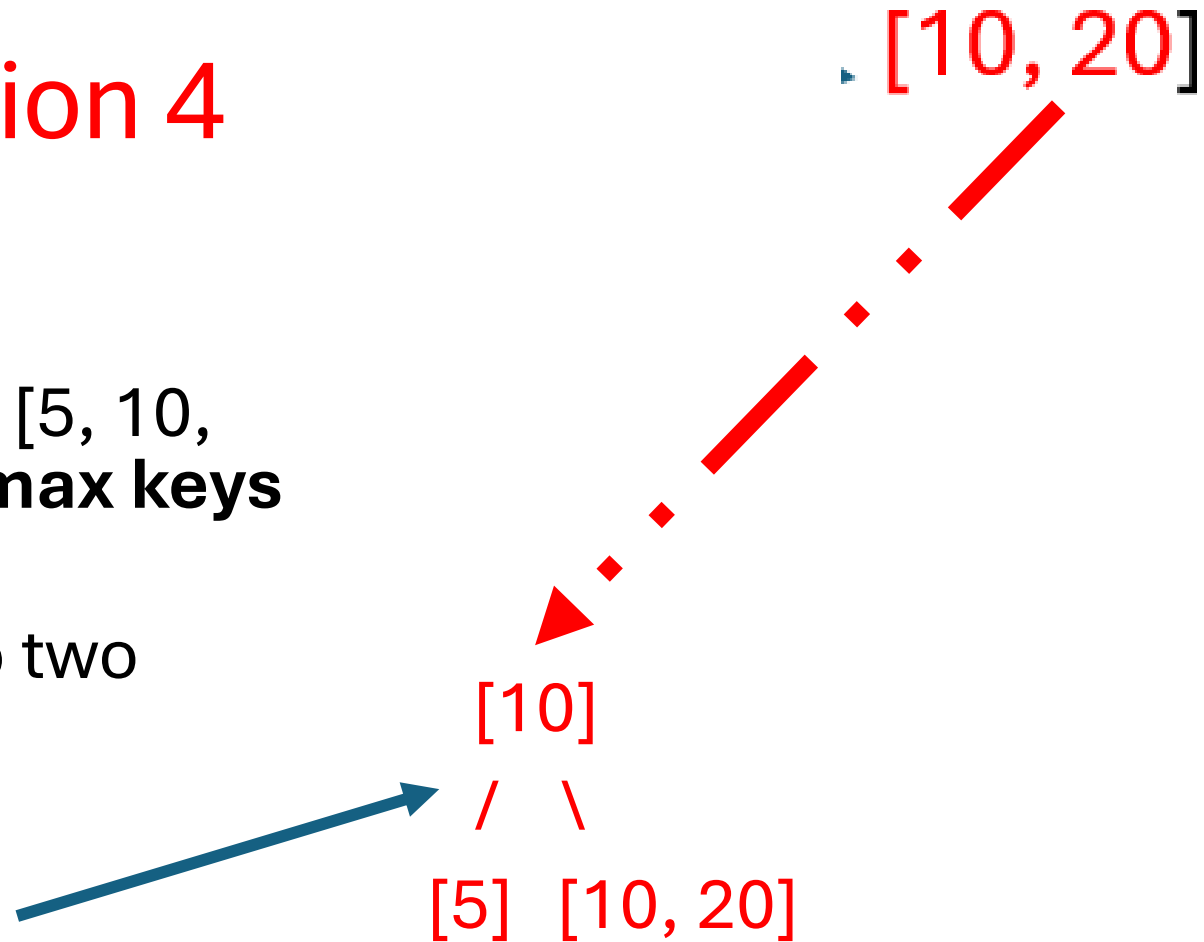
- Add 20 to the same leaf.



# Answer : Question 4

## Step 3: Insert 5

- Add 5 → Now leaf has [5, 10, 20], which **exceeds max keys = 2**.
- So, **split the leaf** into two nodes:
  - Left: [5]
  - Right: [10, 20]
- Push up the **first key of the right node (10)** to create the root.



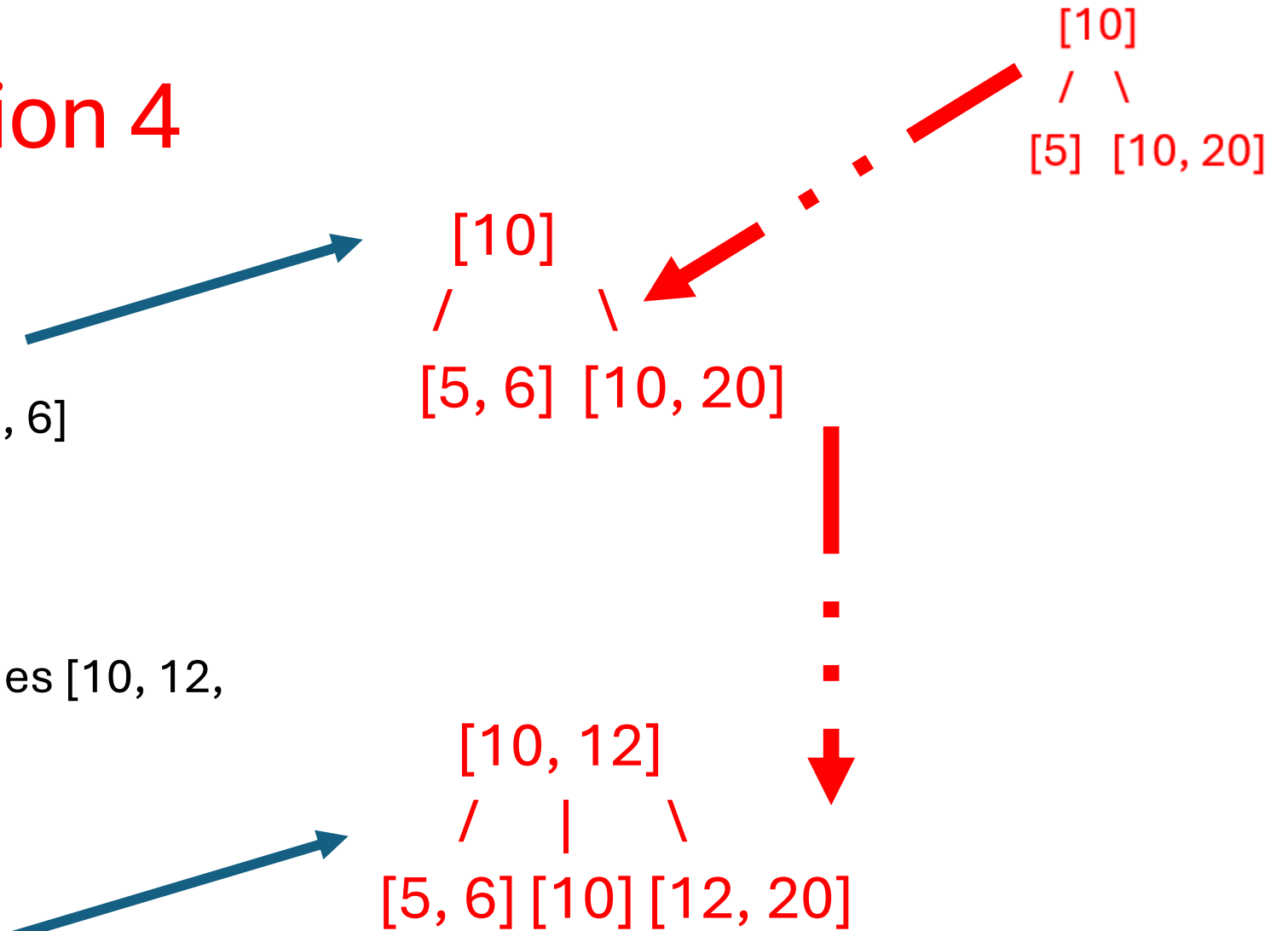
# Answer : Question 4

- **Step 4: Insert 6**

- $6 < 10 \rightarrow$  Goes to [5]
- Insert into [5]  $\rightarrow$  becomes [5, 6]

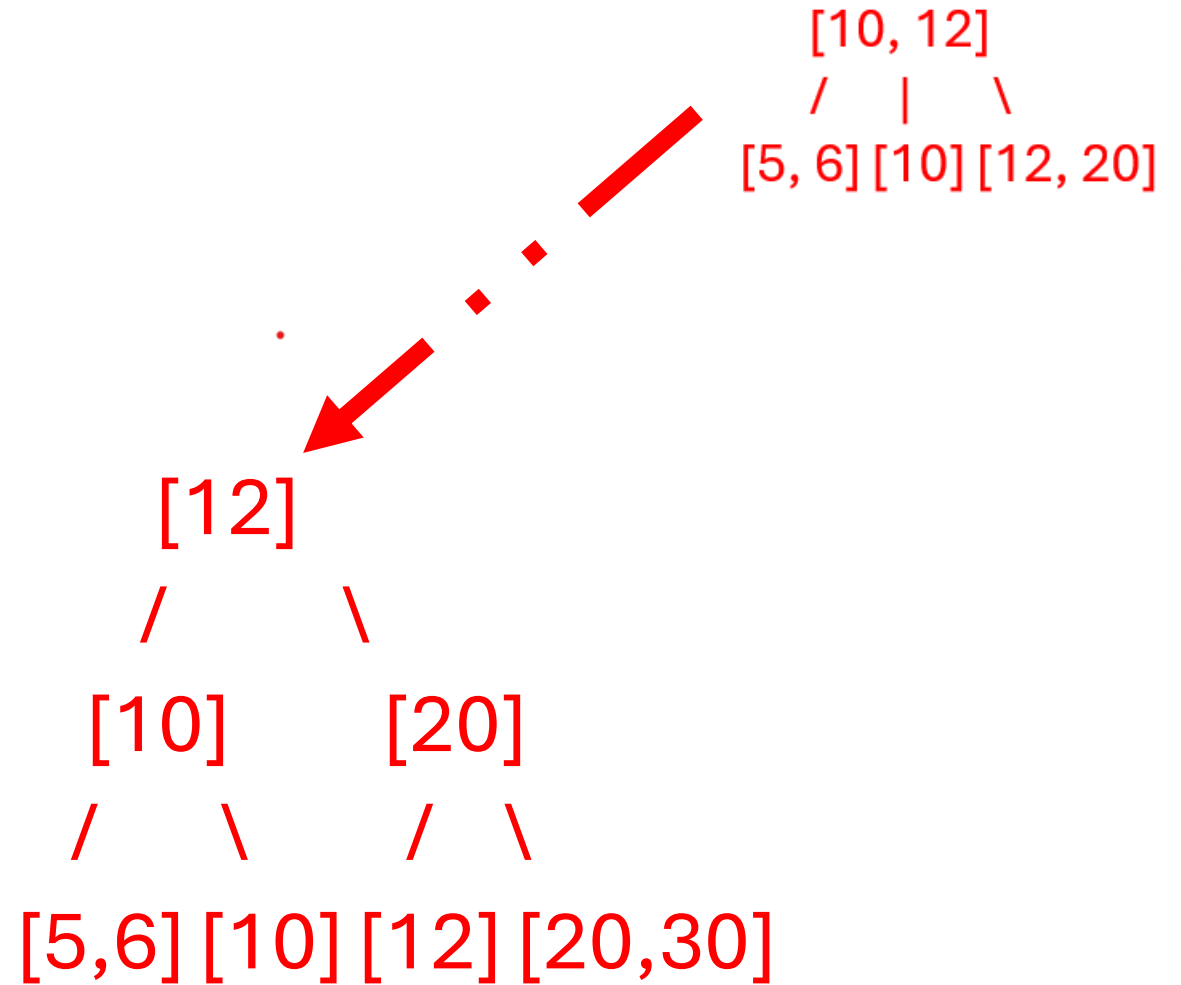
- **Step 5: Insert 12**

- $12 > 10 \rightarrow$  Goes to [10, 20]
- Insert into [10, 20]  $\rightarrow$  becomes [10, 12, 20]  $\rightarrow$  **overflow!**
- Split:
  - Left: [10]
  - Right: [12, 20]
- Promote **12** to parent.
- Now root has [10, 12], with 3 children:



# Answer : Question 4

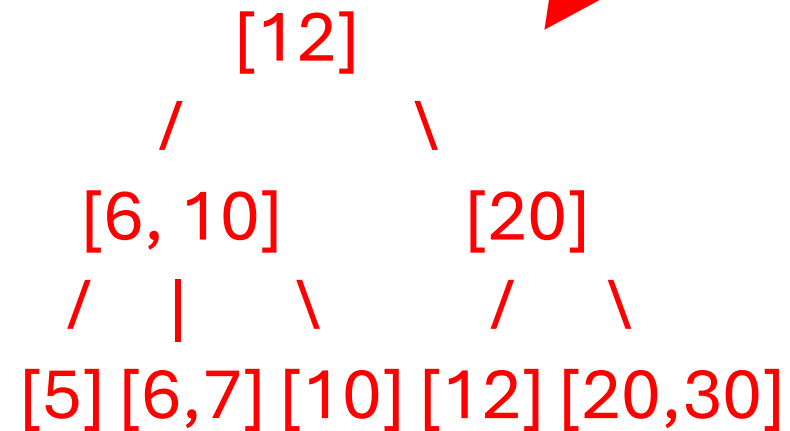
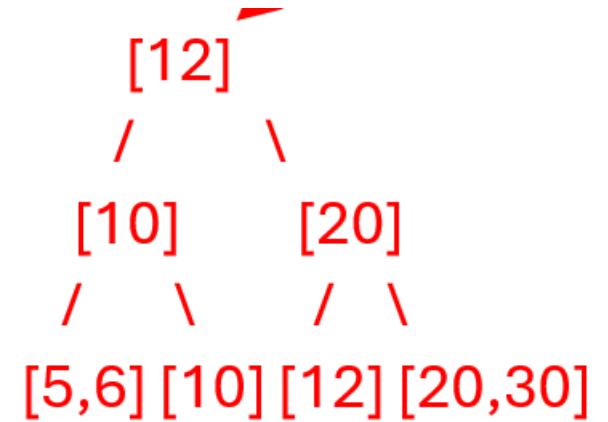
- **Step 6: Insert 30**
- $30 > 12 \rightarrow$  Goes to  $[12, 20]$
- Insert into  $[12, 20] \rightarrow$  becomes  $[12, 20, 30] \rightarrow$  **overflow!**
- Split:
  - Left:  $[12]$
  - Right:  $[20, 30]$
- Promote **20**  $\rightarrow$  Insert into root  $[10, 12] \rightarrow$  becomes  $[10, 12, 20] \rightarrow$  **overflow!**
- Time to **split the root**



# Answer : Question 4

- **Step 7: Insert 7**

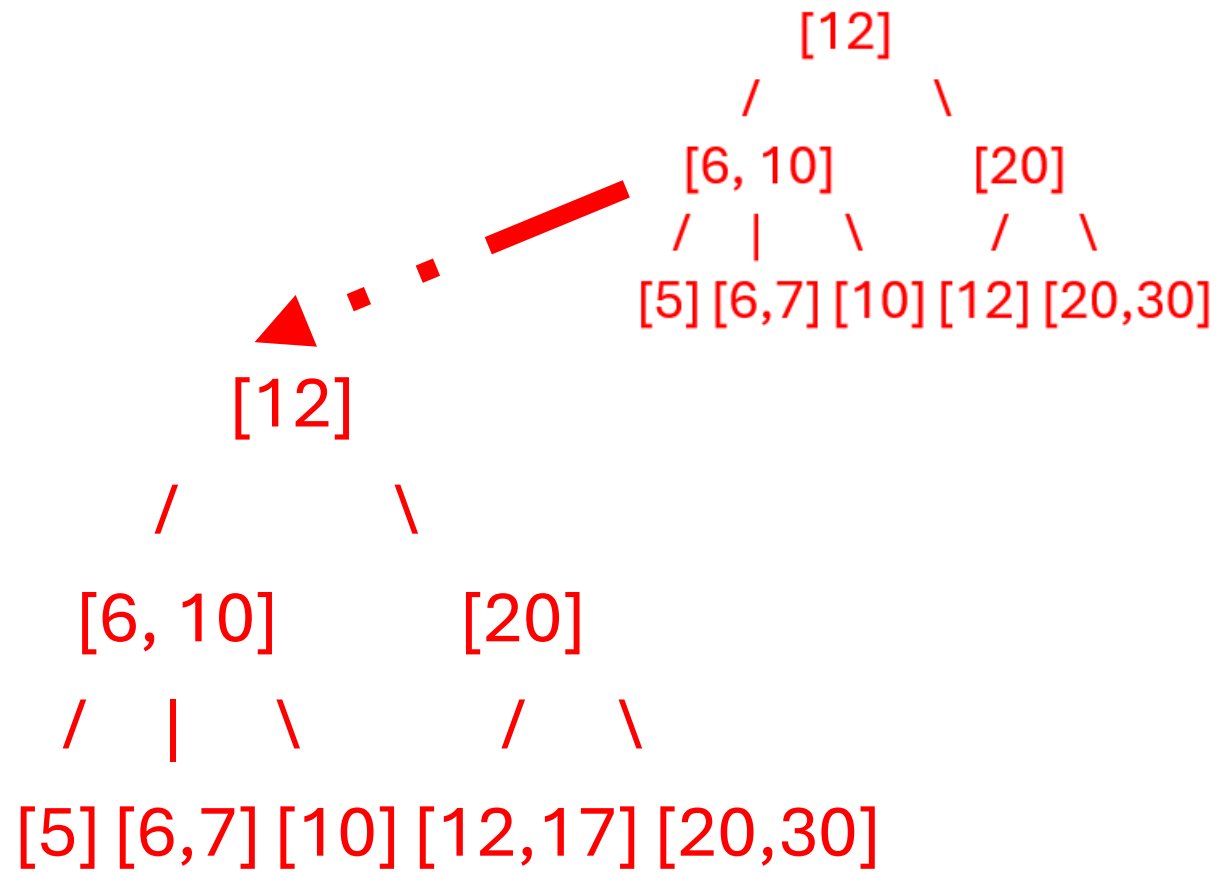
- $7 < 12 \rightarrow$  Left subtree
- $7 < 10 \rightarrow [5, 6]$
- Insert into  $[5, 6] \rightarrow$  becomes  $[5, 6, 7] \rightarrow$  **overflow!**
- Split:
  - Left:  $[5]$
  - Right:  $[6, 7]$
- Promote **6** to parent  $[10]$
- Now update structure:
- Internal node  $[10]$  becomes  $[6, 10]$



# Answer : Question 4

- **Step 8: Insert 17**

- $17 < 20 \rightarrow$  Go to [12] leaf
- Insert 17 into [12]  $\rightarrow$  becomes [12, 17]



- Internal nodes direct the search and hold only keys, not data.
- Leaf nodes contain actual data and are linked sequentially.
- Splits propagate upward, and root split causes tree height increase.
- All data exists at **leaf level**; **internal nodes only guide**.

# Answer : Question 4

(b) Why Are Leaf Nodes Linked in a B+ Tree?

- In a **B+ Tree**, **only the leaf nodes store the actual data** (records). To efficiently **support range queries, sequential access, and fast full-table scans**, **leaf nodes are connected using pointers (linked list fashion)**.
- Each **leaf node** has a **pointer to its right sibling** (and sometimes left).
- This allows traversal of **all data in sorted order without needing to go back up the tree**.